파이썬 완전 백과사전 2권 · 딥러닝 기초 교안

# 5일차

# 남이 학습한 모델 빌려 쓰기

21~25차시 · 책 10장 전이 학습 · 5일 과정의 마지막 날

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/10-transfer-learning

# 오늘의 시간표

| 차시 | 주제 | 책 범위 |
|---|---|---|
| 21 | 사전학습 모델, 전처리, EuroSAT 위성 사진 | 10장 들어가며 ~ 10.3 |
| 22 | 특징 추출: 눈은 빌리고 판단만 새로 | 10.4 |
| 23 | 처음부터 학습과 미세조정 | 10.5 ~ 10.6 |
| 24 | 다른 모델, 다른 입력, 저장하고 불러오기 | 10.7 ~ 10.8 |
| 25 | 실습: 데이터가 적을수록 전이 학습이 빛난다 · 5일 과정 마무리 | 10.9 |

**수업 전 준비 · 한 사람 약 170MB**

EuroSAT 사진 94.3MB와 사전학습 가중치(ResNet18 44.7MB, MobileNetV3-Small 9.8MB, EfficientNet-B0 20.5MB)를 **미리 내려받아 둔다**

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분 (25차시는 과정 마무리 포함)

21차시

# 21

# 사전학습 모델과 이 장의 데이터

책 10장 들어가며 ~ 10.3절

**학습 목표**

- 사전학습 모델을 불러와 몸통과 머리를 가른다
- weights.transforms()로 모델이 배운 전처리를 확인한다
- EuroSAT 위성 사진을 학습·검증·시험으로 나눠 불러온다

# 그림 잘 그리는 사람은 지도도 빨리 읽는다

### 미리 배운 것

사진 120만 장으로 1,000가지 물건을 구별하며 익힌 **사진을 보는 눈**

가장자리, 모서리, 질감, 모양

### 새로 배울 것

"이런 무늬는 숲이라고 부른다"는 **이름 붙이기**뿐

우리 문제: 위성 사진 열 가지 땅

### 같은 ResNet18, 사진 500장

무작위에서 처음부터
**검증 68.1%**

사전학습 모델에서 출발
**검증 93% 넘음**

**전이 학습**(transfer learning): 남이 미리 학습해 둔 **사전학습 모델**의 눈을 빌려 와 우리 문제에 맞는 부분만 새로 학습한다

21차시 · 10장 들어가며 · 숫자는 23차시에서 직접 확인한다(b13_scratch.py 68.1%, b14·b15 최고 93%대)

# 10.1 한 줄로 받는 1,170만 개의 숫자

```
가중치 = models.ResNet18_Weights.DEFAULT        # ImageNet으로 미리 학습한 가중치
모델 = models.resnet18(weights=가중치)           # 처음 한 번은 약 45MB를 내려받는다
print(가중치)
개수 = sum(p.numel() for p in 모델.parameters())
바이트 = sum(p.numel() * p.element_size() for p in 모델.parameters())
print(f"파라미터 {개수:,}개, float32로 {바이트 / 1024**2:.1f}MB")
print(f"모델 카드에 적힌 개수 {가중치.meta['num_params']:,}개, 파일 {가중치.meta['_file_size']}MB")
print("ImageNet 검증 데이터 정확도(top-1):", 가중치.meta["_metrics"]["ImageNet-1K"]["acc@1"], "%")
범주 = 가중치.meta["categories"]
print(f"알아보는 범주 {len(범주)}가지:", 범주[:3], "...", 범주[-2:])
```

```
ResNet18_Weights.IMAGENET1K_V1
파라미터 11,689,512개, float32로 44.6MB
모델 카드에 적힌 개수 11,689,512개, 파일 44.661MB
ImageNet 검증 데이터 정확도(top-1): 69.758 %
알아보는 범주 1000가지: ['tench', 'goldfish', 'great white shark'] ... ['ear', 'toilet tissue']
```

21차시 · 10.1 사전학습 모델 불러오기 · 예제 10-transfer-learning/b01_load.py (핵심 부분)

# 10.1 어떤 모델이 있을까

```
이름들 = models.list_models(module=models)     # torchvision의 사진 분류 모델 목록
w = models.get_model_weights(이름).DEFAULT   # 가중치 정보만 읽는다. 내려받지 않는다
```

| 모델 | 파라미터 | 파일 | ImageNet top-1 | 이런 때 |
|---|---|---|---|---|
| MobileNetV3-Small | 250만 | 9.8MB | 67.7% | 휴대폰·작은 기기에서 빠르게 |
| **ResNet18** | 1,170만 | 44.7MB | 69.8% | 공부용, 무난한 출발점 |
| EfficientNet-B0 | 530만 | 20.5MB | 77.7% | 크기 대비 정확도가 좋은 것 |
| ResNet50 | 2,560만 | 97.8MB | 80.9% | 서버에서 더 높은 정확도 |
| ViT-B/16 | 8,660만 | 330.3MB | 81.1% | 트랜스포머 구조, 큰 데이터 |

크기와 정확도는 꼭 비례하지 않고, ImageNet에서 잘한다고 **우리 데이터에서도** 잘한다는 보장은 없다

# 10.1 몸통과 머리

```
x = torch.zeros(1, 3, 224, 224)   # 사진 1장, 3색, 224 × 224
```

```
conv1    Conv2d                 9,408개 → (1, 64, 112, 112)
bn1      BatchNorm2d              128개 → (1, 64, 112, 112)
relu     ReLU                       0개 → (1, 64, 112, 112)
maxpool  MaxPool2d                  0개 → (1, 64, 56, 56)
layer1   Sequential              147,968개 → (1, 64, 56, 56)
layer2   Sequential              525,568개 → (1, 128, 28, 28)
layer3   Sequential            2,099,712개 → (1, 256, 14, 14)
layer4   Sequential            8,393,728개 → (1, 512, 7, 7)
avgpool  AdaptiveAvgPool2d          0개 → (1, 512, 1, 1)
fc       Linear                513,000개 → (1, 1000)
Linear(in_features=512, out_features=1000, bias=True)
```

### 몸통 · conv1 ~ avgpool

사진을 **특징 벡터 512개**로 바꾸는 눈. 다른 문제에도 쓸모가 있다

### 머리 · fc

512 × 1,000 + 1,000 = 513,000개. ImageNet 범주에 맞춘 것이라 **새것으로 바꾼다**

# 10.2 모델이 배운 기준 그대로

```
전처리 = models.ResNet18_Weights.DEFAULT.transforms()
print(전처리)
```

```
ImageClassification(
    crop_size=[224]
    resize_size=[256]
    mean=[0.485, 0.456, 0.406]
    std=[0.229, 0.224, 0.225]
    interpolation=InterpolationMode.BILINEAR
)
원본: (64, 64) RGB
전처리 후: (3, 224, 224) torch.float32
풀어 쓴 것과 같은가: True
```

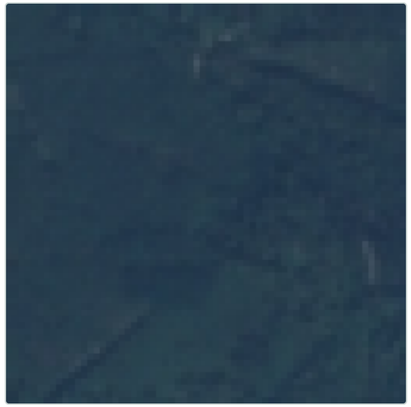
### 10차시의 v2 변환으로 풀어 쓰면

```
v2.ToImage()
v2.Resize(256)      # 짧은 변을 256으로
v2.CenterCrop(224) # 가운데 224 × 224
v2.ToDtype(torch.float32, scale=True)
v2.Normalize(mean=[0.485, 0.456, 0.406],
             std=[0.229, 0.224, 0.225])
```

크기는 바꿔도 된다. 그러나 **평균과 표준편차만은 반드시 ImageNet 값**

21차시 · 10.2 입력 전처리 맞추기 · 예제 10-transfer-learning/b04_transforms.py (핵심 부분)

# 10.2 ImageNet의 눈으로 위성 사진 보기



**숲 Forest**

dugong 6%
velvet 4%
Petri dish 3%



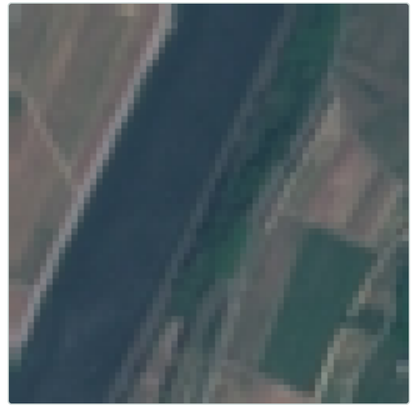
**고속도로 Highway**

safety pin 11%
hand-held computer 10%
reel 4%



**주거지 Residential**

handkerchief 13%
stole 6%
terrapin 6%



**강 River**

Petri dish 4%
Windsor tie 3%
hand-held computer 3%



**바다·호수 SeaLake**

space shuttle 3%
nematode 3%
flagpole 3%

이 모델은 **눈은 좋지만 우리 문제의 단어를 모른다**. 눈(몸통)은 남기고 단어를 고르는 머리만 다시 가르치면 된다

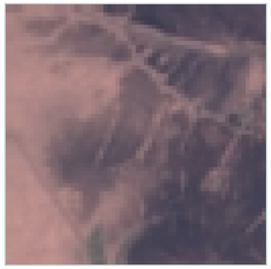
# 10.3 EuroSAT: 64 × 64 위성 사진 27,000장



**0 경작지**
3,000장



**1 숲**
3,000장



**2 초지**
3,000장



**3 고속도로**
2,500장



**4 산업지역**
2,500장



**5 목초지**
2,000장



**6 과수원**
2,500장



**7 주거지**
3,000장



**8 강**
2,500장



**9 바다·호수**
3,000장

### 평균 색 R·G·B (클래스마다 100장)

| 클래스 | 평균 색 |
|---|---|
| 숲 | [39, 64, 76] |
| 바다·호수 | [45, 69, 86] |
| 고속도로 | [89, 97, 103] |
| 주거지 | [95, 101, 110] |
| 초지 | [100, 99, 104] |
| 산업지역 | [127, 129, 137] |

색이 거의 같은 짝은
**무늬와 모양**으로 가려야 한다

21차시 · 10.3 이 장의 데이터: EuroSAT 위성 사진 · 예제 10-transfer-learning/b06_eurosat.py (사진은 클래스마다 첫 장)

# 10.3 공통 준비 코드: 전이도구.py

| 함수 | 하는 일 | 처음 본 곳 |
|---|---|---|
| 장치_고르기() | cuda → mps → cpu 순서로 고른다 | 8차시 |
| 유로샛_불러오기(구간, 클래스당) | 클래스마다 같은 수, 겹치지 않는 구간 (학습 0 · 검증 1000 · 시험 1500번째부터) | 11차시 분할 |
| 변환_만들기(크기=112, 증강) | Resize → (뒤집기) → 0~1 → **ImageNet 기준** 정규화 | 10차시 |
| 사진_데이터셋, 로더_만들기 | 꺼낼 때마다 변환, 배치 64, 섞는 순서 씨앗 고정 | 9·11차시 |
| 한_에폭_학습, 정확도 | 표준 학습 루프와 평가 루프 | 7·8차시 |
| 특징_뽑기, 머리_학습 | 얼린 몸통으로 특징을 뽑고 마지막 층 하나만 학습 | 22차시에서 |

새로운 건 거의 없다. 6장 학습 루프와 7장 Dataset·transforms·분할을 **이 장의 데이터에 맞게 모은 것**

# 10.3 학습·검증·시험으로 나눠 불러오기

```
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
시험_사진, 시험_정답 = 유로샛_불러오기("시험", 100)
x, y = next(iter(로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)))
```

```
학습: (500, 3, 64, 64) torch.uint8, 클래스별 [50, 50, 50, 50, 50, 50, 50, 50, 50, 50]
검증: (1000, 3, 64, 64) torch.uint8, 클래스별 [100, 100, 100, 100, 100, 100, 100, 100, 100, 100]
시험: (1000, 3, 64, 64) torch.uint8, 클래스별 [100, 100, 100, 100, 100, 100, 100, 100, 100, 100]
학습 사진이 차지하는 메모리 5.9MB
배치 하나: (64, 3, 112, 112) torch.float32 [8, 9, 8, 8, 9, 5, 4, 3]
채널별 평균: ['-0.66', '-0.37', '-0.05'] 표준편차: ['0.86', '0.55', '0.44']
```

27,000장 중 **500장만** 학습에 쓴다. 정답 붙은 사진이 몇백 장뿐인 현장이 흔하다

정규화했는데 평균이 0이 아니다. **틀린 걸까?** 22차시에서 확인한다

# 직접 실행 10분

1. 10-transfer-learning 폴더에서 b01_load.py, b03_structure.py를 실행하고 몸통과 머리의 경계를 출력에서 찾는다
2. b05_imagenet_predict.py를 복사해 k 목록을 [0, 2, 4, 5, 6]으로 바꾼다. 경작지·초지·산업지역·목초지·과수원을 ImageNet 모델은 뭐라고 부를까?
3. b03의 가짜 사진을 112 × 112로 바꿔 실행한다. layer4와 avgpool을 지난 모양은?

```
cd examples/vol2/10-transfer-learning && uv run python b03_structure.py
```

# ResNet18의 fc를 nn.Linear(512, 10)으로 바꾸면, 새 머리의 파라미터는 몇 개일까요?

Q

가 5,120개

나 5,130개

다 513,000개

라 11,181,642개

# 21차시 정리

**사전학습 모델**: models.resnet18(weights=ResNet18_Weights.DEFAULT). meta에 파라미터 수, 파일 크기, 정확도, 범주

**몸통**은 사진을 특징 512개로 바꾸는 눈, **머리**(fc)는 원래 문제에 맞춘 마지막 층. 전이 학습은 머리를 바꾸는 데서 시작

**weights.transforms()**를 출력해 확인한다. 크기는 바꿔도 평균·표준편차는 ImageNet 값

다음 차시: 몸통을 얼리고 머리만 학습한다 · 특징 추출

22차시

22

# 특징 추출: 눈은 빌리고 판단만 새로

책 10.4절

**학습 목표**

- 몸통을 얼리고 머리만 학습하는 특징 추출을 짠다
- 특징을 한 번만 뽑아 두는 요령과 그 대가를 말한다
- 배치 정규화 함정, 전처리 실수, 입력 크기를 숫자로 확인한다

# 10.4 몸통 얼리기

```
모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in 모델.parameters():
    p.requires_grad_(False)                  # 몸통 전체를 얼린다(6장 6.4절)
모델.fc = nn.Linear(모델.fc.in_features, 10)  # 새로 만든 층은 얼어 있지 않다

전체 = sum(p.numel() for p in 모델.parameters())
학습_대상 = sum(p.numel() for p in 모델.parameters() if p.requires_grad)
print(f"전체 {전체:,}개 중 학습 대상 {학습_대상:,}개 ({학습_대상 / 전체:.3%})")
# 옵티마이저에는 학습 대상만 넘긴다
옵티마이저 = torch.optim.Adam(모델.fc.parameters(), lr=1e-3)
```

```
전체 11,181,642개 중 학습 대상 5,130개 (0.046%)
학습 대상 이름: ['fc.weight', 'fc.bias']
conv1.weight.grad: None
fc.weight.grad 모양: (10, 512)
옵티마이저가 맡은 텐서 수: 2
```

**순서가 중요하다**

얼리고 **그다음에** 머리를 바꾼다. 거꾸로 하면 새 머리까지 얼어 아무것도 학습되지 않는다

# 10.4 특징을 한 번만 뽑아 두기

```
몸통 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
몸통.fc = nn.Identity()  # 마지막 층을 "그대로 통과"로 바꾸면 특징 추출기
몸통.requires_grad_(False)
학습_특징, 학습_정답 = 특징_뽑기(몸통, 로더_만들기(학습_사진, 학습_정답), 장치)
검증_특징, 검증_정답 = 특징_뽑기(몸통, 로더_만들기(검증_사진, 검증_정답), 장치)
print("특징:", tuple(학습_특징.shape), tuple(검증_특징.shape))

# 이제 학습은 특징 512개 → 10클래스, 층 하나뿐이다
torch.manual_seed(0)
머리 = nn.Linear(512, 10)
옵티마이저 = torch.optim.Adam(머리.parameters(), lr=1e-3)
for 에폭 in range(1, 101):
    옵티마이저.zero_grad()
    손실 = nn.functional.cross_entropy(머리(학습_특징), 학습_정답)
    손실.backward()
    옵티마이저.step()
```

```
특징: (500, 512) (1000, 512)
```

**몸통 통과 횟수**

매번 돌리면 500 × 100 = **50,000번**
뽑아 두면 학습·검증 **1,500번**

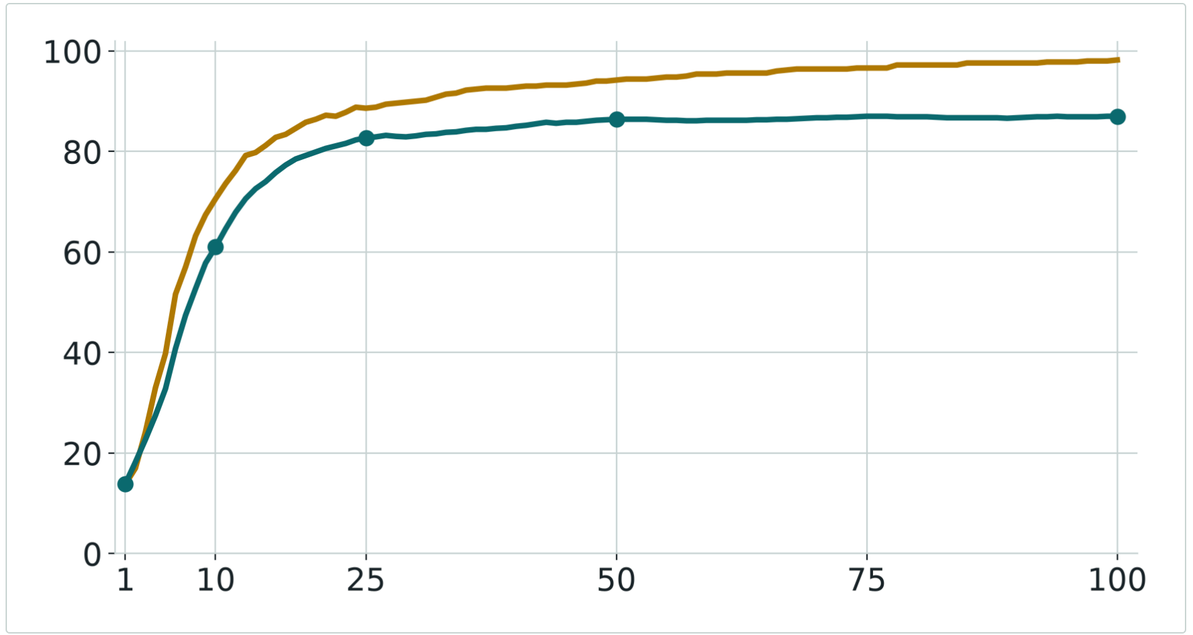
**대가**

특징이 고정되니 **데이터 증강을 쓸 수 없다**

# 10.4 층 하나, 사진 500장으로 87.0%

```
에폭   1: 손실 2.3806, 학습 13.8%, 검증 13.9%
에폭  10: 손실 1.6413, 학습 70.6%, 검증 61.0%
에폭  25: 손실 0.9267, 학습 88.6%, 검증 82.7%
에폭  50: 손실 0.4790, 학습 94.2%, 검증 86.4%
에폭 100: 손실 0.2296, 학습 98.2%, 검증 87.0%
```

- 무작위로 찍으면 10%인 문제에서 **열에 아홉 가까이**
- 학습 98.2% 대 검증 87.0%: 과적합이 조금 보이지만 검증은 50에폭 뒤에도 내려가지 않는다




— 학습 정확도 · — 검증 정확도 (● 출력한 에폭) · 가로는 에폭, 세로는 %

22차시 · 10.4 특징을 한 번만 뽑아 두기 · 예제 10-transfer-learning/b09_features.py (출력, 그림은 같은 설정으로 에폭마다 다시 잰 것)

# 10.4 얼렸는데도 바뀌는 것: 배치 정규화

```
모델.requires_grad_(False)                    # 전부 얼렸다
모델.fc = nn.Identity()
...
모델.train()                                  # 습관대로 train 모드로 몇 배치 돌리면
with torch.no_grad():
    for _ in range(5):
        모델(x)
```

```
학습 대상 파라미터: 0
bn1 기록 평균(전): ['0.003', '-0.026', '0.000', '-0.085']
bn1 기록 평균(후): ['-0.006', '0.070', '-0.000', '0.221']
같은 사진의 특징이 바뀌었나? 최대 차이 2.841, 코사인 유사도 평균 0.945
```

기록 평균·분산은 기울기로 고치는 파라미터가 아니라 **버퍼**. requires_grad와 상관없이 train 모드에서 바뀐다

22차시 · 10.4 얼렸는데도 바뀌는 것 · 예제 10-transfer-learning/b10_bn_trap.py (핵심 부분)

# 10.4 해결: 배치 정규화만 평가 모드로

```
# 해결: train 모드로 돌리더라도 배치 정규화 층만은 평가 모드로 묶어 둔다
def 배치정규화_고정(모델: nn.Module) -> None:
    for 층 in 모델.modules():
        if isinstance(층, nn.BatchNorm2d):
            층.eval()

새모델.train()
배치정규화_고정(새모델)
```

```
고정한 bn1 기록 평균: ['0.003', '-0.026', '0.000', '-0.085']
```

| 서로 다른 스위치 | 무엇을 정하나 | 얼린 몸통에서 |
|---|---|---|
| requires_grad | 기울기로 고칠까 | False: 가중치는 그대로 |
| train() / eval() | 배치 정규화 기록, 드롭아웃 | train이면 기록이 바뀐다 |

# 10.4 전처리를 틀리면




```
v2.ToDtype(torch.float32, scale=True)   # 0~1 실수로
v2.ToDtype(torch.float32)               # scale을 빼먹으면 0~255 그대로
```

코드는 **오류 없이** 돌아가고 정확도만 조용히 떨어진다

# 10.4 입력 크기는 얼마로 할까

```
몸통.eval()                          # 세는 동안에도 배치 정규화 기록이 바뀌지 않게
with FlopCounterMode(display=False) as 계수기:   # 사진 한 장에 드는 곱셈·덧셈 수 세기
    몸통.cpu()(torch.zeros(1, 3, 크기, 크기))
```

| 입력 크기 | layer4 지도 | 한 장에 | 검증 정확도 |
|---|---|---|---|
| 64 × 64 | 2 × 2 | 0.30 GFLOP | 84.3% |
| **112 × 112** | 4 × 4 | 0.97 GFLOP | 87.0% |
| 160 × 160 | 5 × 5 | 1.85 GFLOP | 89.1% |
| 224 × 224 | 7 × 7 | 3.63 GFLOP | 89.3% |

**112 → 224**

정확도 +2.3%p

계산량 3.7배

정확도와 계산 비용의 저울질. 실험을 많이 돌리는 이 장은 112, 최종 모델은 160이나 224도 좋은 선택

# 직접 실행 10분

1. b09_features.py를 실행해 에폭 100의 검증 87.0%를 확인한다
2. b09를 복사해 유로샛_불러오기("학습", 50)의 50을 20으로, 다음엔 100으로 바꾼다. 에폭 100의 학습·검증 정확도는?
3. b11_preprocess.py를 실행하기 **전에** 네 전처리의 순위를 예상해 적고, 실행해 맞춰 본다

```
cd examples/vol2/10-transfer-learning && uv run python b09_features.py
```

22차시 · 직접 실행 · b09 2초, 복사본 2~3초, b11 3초 (MPS 기준, CPU에서는 특징 뽑기가 수십 초)

# 파라미터를 모두 얼린 몸통을 train() 모드로 돌렸더니 특징이 바뀌었습니다. 무엇이 바뀐 걸까요?

Q

**가** 합성곱 층의 가중치

**나** 배치 정규화의 기록 평균·분산

**다** 새로 붙인 머리의 가중치

**라** 아무것도 안 바뀌었다

# 22차시 정리

**특징 추출**: 몸통을 얼리고(requires_grad_(False)) 머리만 학습. fc = nn.Identity()로 특징을 한 번 뽑아 두면 빠르지만 증강은 못 쓴다

**배치 정규화 함정**: 얼린 몸통도 train()이면 기록이 바뀐다. 얼린 부분은 eval()로 묶는다

**전처리**는 ImageNet 평균·표준편차와 scale=True. 크기는 정확도와 계산량의 저울질(112 → 224: +2.3%p, 3.7배)

다음 차시: 처음부터 학습과 비교하고, 몸통까지 조금씩 고친다 · 미세조정

23차시

23

# 처음부터 학습과 미세조정

책 10.5 ~ 10.6절

**학습 목표**

- 같은 구조를 처음부터 학습한 결과와 비교한다
- 미세조정 학습률을 고르고, 몸통과 머리에 다른 학습률을 준다
- 어디까지 풀지 정하고, 어느 층이 얼마나 바뀌는지 잰다

# 10.5 처음부터 학습하면 어떨까

```
모델 = models.resnet18(weights=None, num_classes=10).to(장치)   # 같은 구조, 무작위 가중치
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
```




— 학습 · — 검증 · - - 특징 추출 87.0% · 가로 에폭, 세로 %

**30에폭 뒤 검증**

**68.1%**

최고는 25에폭 69.9%
학습은 82.4%

1,100만 개를 모두 학습했는데
특징 추출(87.0%)보다 **20%p 가까이** 낮다

# 10.5 위성 사진은 ImageNet과 다른데 왜 도움이 될까

### 앞부분

가장자리, 모서리, 색의 변화, 질감

어떤 사진에나 있는 **기본 재료**라 그대로 쓸모가 있다

### 뒷부분

"개의 귀", "자동차 바퀴"

ImageNet에 특화된 특징. 우리 데이터에 맞게 **고쳐 주면** 더 좋아진다

### 아주 다른 데이터

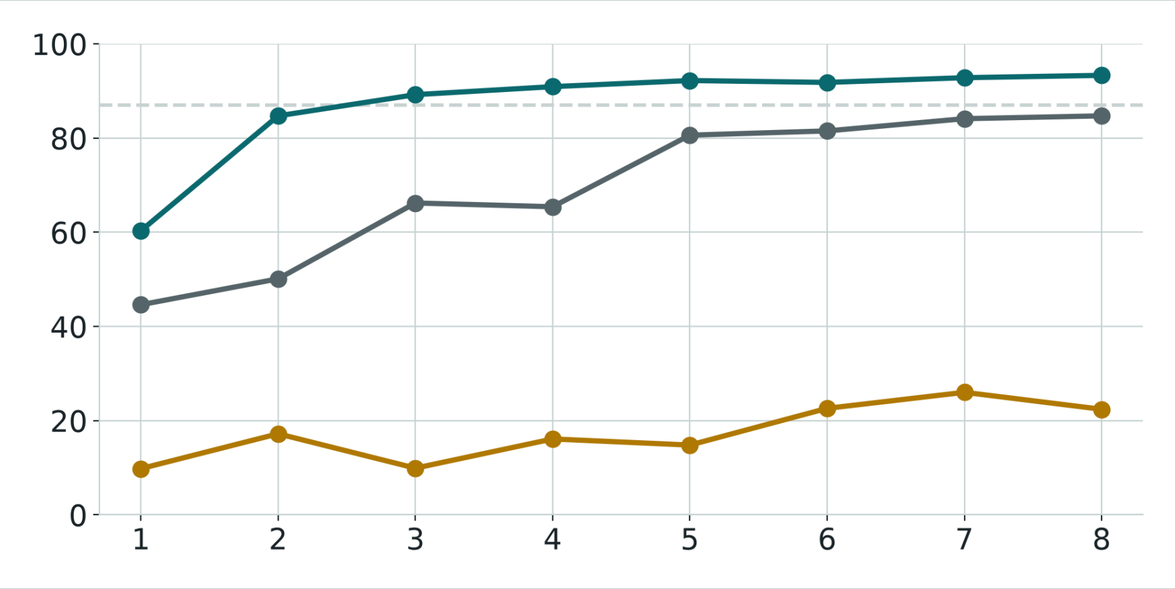
의료 영상, 전파 신호

효과가 작을 수 있다. 처음부터 학습한 결과와 **비교하는 습관**

뒷부분을 우리 데이터에 맞게 조금씩 고치는 방법이 다음 절의 **미세조정**

# 10.6 미세조정: 학습률이 전부다

```
lr=0.01: 에폭별 검증 정확도 10% 17% 10% 16% 15% 23% 26% 22%
lr=0.001: 에폭별 검증 정확도 45% 50% 66% 65% 81% 82% 84% 85%
lr=0.0001: 에폭별 검증 정확도 60% 85% 89% 91% 92% 92% 93% 93%
```




● lr=0.0001 · ● lr=0.001 · ● lr=0.01 · 점선 특징 추출 87.0%

0.01: 큰 걸음으로 휘저어 물려받은 눈을 거의 다 잊었다

0.001: 처음에 흐려 놓고 다시 배우느라 85%

0.0001: 둘째 에폭 85%, 끝내 93%

23차시 · 10.6 미세조정: 몸통까지 조금씩 고치기 · 예제 10-transfer-learning/b14_finetune_lr.py (출력과, 같은 설정으로 다시 돌려 그린 그림)

# 10.6 층마다 다른 학습률

```
몸통_파라미터 = [p for 이름, p in 모델.named_parameters() if not 이름.startswith("fc.")]
옵티마이저 = torch.optim.Adam([
    {"params": 몸통_파라미터, "lr": 1e-4},          # 이미 잘 배운 부분은 조심스럽게
    {"params": 모델.fc.parameters(), "lr": 1e-3},   # 처음 배우는 머리는 크게
])
for i, 그룹 in enumerate(옵티마이저.param_groups):
    개수 = sum(p.numel() for p in 그룹["params"])
    print(f"그룹 {i}: 텐서 {len(그룹['params'])}개, 파라미터 {개수:,}개, lr={그룹['lr']:g}")
```

```
그룹 0: 텐서 60개, 파라미터 11,176,512개, lr=0.0001
그룹 1: 텐서 2개, 파라미터 5,130개, lr=0.001
전부 1e-4: 에폭별 검증 정확도 61.7% 84.0% 89.4% 90.6% 92.2% 92.7% 92.8% 92.6%
몸통 1e-4 + 머리 1e-3: 에폭별 검증 정확도 66.7% 85.1% 87.7% 90.4% 91.8% 92.8% 93.2% 93.4%
```

첫 에폭부터 앞서고(61.7 → 66.7) 8에폭 뒤 0.8%p 차이. 씨앗만 바꿔도 흔들릴 크기지만, 머리가 무작위에서 시작하니 **나누는 편이 자연스럽다**

23차시 · 10.6 층마다 다른 학습률 · 예제 10-transfer-learning/b15_param_groups.py (핵심 부분)

# 10.6 어디까지 풀까

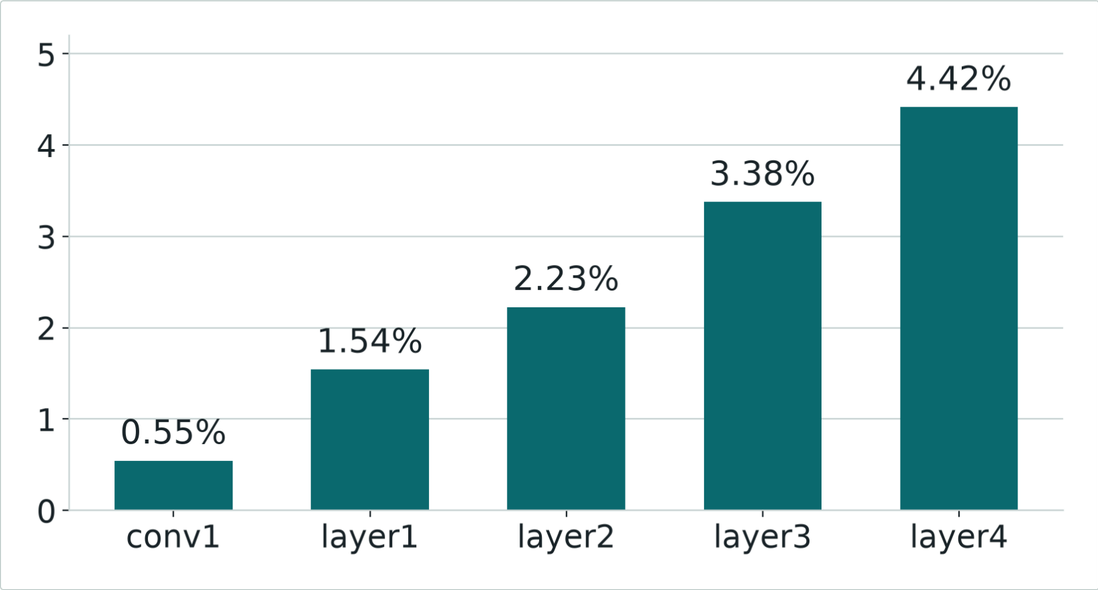
```
순서 = ["conv1", "bn1", "layer1", "layer2", "layer3", "layer4"]   # 입력에 가까운 쪽부터
for 이름 in 얼린_부분:
    모델.get_submodule(이름).requires_grad_(False)
...
    for 이름 in 얼린_부분:  # 얼린 부분은 배치 정규화 기록도 그대로 둔다(10.4절)
        모델.get_submodule(이름).eval()
```

| 푼 부분 | 학습 대상 | 검증 정확도 (8에폭) |
|---|---|---|
| 없음 (머리만) | 5,130 | 83.6% |
| layer4 | 8,398,858 | 91.4% |
| layer3, layer4 | 10,498,570 | 92.7% |
| layer1 ~ layer4 | 11,172,106 | 93.1% |
| 전부 (conv1, bn1까지) | 11,181,642 | 93.1% |

**layer4 하나만** 풀어도 8%p 가까이 뛴다. 앞부분까지 풀어도 거의 같다

# 10.6 무엇이 얼마나 바뀌었을까

```
처음 = copy.deepcopy(모델.state_dict())           # 미세조정 전 가중치를 복사해 둔다
# 부분마다 가중치가 처음보다 몇 % 바뀌었나:  ‖나중 - 처음‖  /  ‖처음‖
print(f"{부분:7} 변화율 {(후 - 전).norm() / 전.norm():.2%}")
```




세로: 처음 가중치 대비 바뀐 양(%) · 검증 정확도 92.6%

같은 학습률인데 뒤로 갈수록 **8배**
앞부분의 기본 특징은 위성 사진에도 맞았다

가장 많이 바뀐 layer4도 **4.42%**
물려받은 가중치는 거의 그대로 남는다

23차시 · 10.6 무엇이 얼마나 바뀌었을까 · 예제 10-transfer-learning/b17_changed.py (핵심 부분, 그림은 같은 설정으로 다시 돌려 그린 것)

# 얼릴까 풀까: 실무 요령

| 이럴 때 | 이렇게 |
|---|---|
| 데이터가 아주 적거나 ImageNet과 비슷한 사진 | 많이 얼린다 (특징 추출에 가깝게) |
| 데이터가 넉넉하거나 ImageNet과 다른 사진 | 많이 푼다 (미세조정에 가깝게) |
| 계산 자원이 빠듯하다 | 뒷부분만 푼다. 얼린 부분은 기울기를 안 구해 빠르고 메모리도 덜 든다 |
| 미세조정 학습률 | 처음부터 학습할 때보다 **10배쯤 작게**, 머리는 따로 크게 |

어느 쪽이든 마지막 판단은 **검증 데이터**로

# 직접 실행 10분

1. b14_finetune_lr.py를 복사해 학습률 목록을 [3e-4, 3e-5]로 바꾼다. 실행하기 전에 어느 쪽이 93%에 더 가까울지 예상해 적는다
2. 실행을 걸어 둔 동안 b17_changed.py의 출력을 보고, 앞부분일수록 덜 바뀐 이유를 한 문장으로 쓴다
3. 두 결과를 lr=0.0001(93%)과 비교해, "10배쯤 작게"가 왜 출발점인지 말해 본다

```
for 학습률 in [3e-4, 3e-5]:
```

23차시 · 직접 실행 · 복사본 22초, b17 6초 (MPS 기준, CPU만 있으면 강사 화면으로)

# 사전학습 ResNet18 전체를 lr=0.01로 미세조정하면 어떻게 됐나요?

Q

**가** 93%로 가장 좋았다

**나** 특징 추출과 같은 87%

**다** 10~26%를 오갔다

**라** 오류가 나서 멈췄다

# 23차시 정리

같은 ResNet18, 사진 500장: **처음부터 68.1%** · 특징 추출 87.0% · **미세조정 93%대**

**미세조정**의 학습률은 10배쯤 작게. 몸통과 머리는 param_groups로 학습률을 따로

뒷부분일수록 많이 바뀐다(conv1 0.55% → layer4 4.42%). 계산이 빠듯하면 **뒷부분만** 푼다

다음 차시: 다른 모델로 바꿔 끼우고, 흑백 사진을 넣고, 저장하고 불러온다

24차시

24

# 다른 모델, 다른 입력, 저장

책 10.7 ~ 10.8절

**학습 목표**

- 어떤 모델이든 마지막 nn.Linear를 찾아 머리를 바꾼다
- 몸통을 바꿔 끼워 비교하고, 흑백 사진을 넣는 두 방법을 안다
- 머리를 바꾼 모델을 저장하고 다시 불러온다

# 10.7 머리 바꾸기를 일반화하기

```
def 머리_바꾸기(모델: nn.Module, 클래스_수: int) -> str:
    """모델에 등록된 마지막 nn.Linear를 찾아 클래스_수만큼 내는 새 층으로 바꾼다."""
    이름, 마지막 = [(n, m) for n, m in 모델.named_modules() if isinstance(m, nn.Linear)][-1]
    부모_이름, _, 자식_이름 = 이름.rpartition(".")   # "classifier.3" → "classifier", "3"
    setattr(모델.get_submodule(부모_이름), 자식_이름, nn.Linear(마지막.in_features, 클래스_수))
    return 이름

for 이름 in ["resnet18", "mobilenet_v3_small", "efficientnet_b0", "vit_b_16"]:
    모델 = models.get_model(이름, weights=None)   # 구조만 본다
    바꾼_곳 = 머리_바꾸기(모델, 10)
```

### ① 찾기

named_modules()에서 nn.Linear만 골라 마지막 것

### ② 나누기

rpartition(".")으로 부모 이름과 자식 이름

### ③ 갈아 끼우기

get_submodule로 부모를 꺼내 setattr

# 10.7 모델마다 머리가 다른 곳에 있다

| 모델 | 바꾼 마지막 층 | 들어오는 특징 | 출력 |
|---|---|---|---|
| resnet18 | `fc` | 512 | (1, 10) |
| mobilenet_v3_small | `classifier.3` | 1,024 | (1, 10) |
| efficientnet_b0 | `classifier.1` | 1,280 | (1, 10) |
| vit_b_16 | `heads.head` | 768 | (1, 10) |

"등록한 순서의 마지막 nn.Linear가 머리"라는 가정은 대부분 맞지만 직접 만든 모델에서는 아닐 수 있다. 처음 쓰는 모델은 **print(모델)**로 한 번 훑어본다

# 10.7 몸통 바꿔 끼우기

```
모델 = models.get_model(이름, weights="DEFAULT")   # 처음 한 번은 가중치를 내려받는다
setattr(모델.get_submodule(부모), 자식, nn.Identity())   # 마지막 층을 떼면 특징 추출기
```

| 모델 | 몸통 | 특징 | ImageNet top-1 | 우리 검증 |
|---|---|---|---|---|
| MobileNetV3-Small | 150만 | 1,024 | 67.7% | 85.5% |
| **ResNet18** | 1,120만 | 512 | 69.8% | **87.0%** |
| EfficientNet-B0 | 400만 | 1,280 | **77.7%** | 85.6% |

원래 문제에서 잘한 순서가 **새 문제에서도 유지된다는 보장은 없다**

MobileNet은 몸통이 7분의 1인데 차이는 1.5%p. 작은 기기라면 이쪽

# 10.7 흑백 사진을 넣고 싶다면

```
# 방법 1: 흑백 한 채널을 세 번 복사해 컬러처럼 넣는다
with torch.no_grad():
    출력1 = 모델(흑백.repeat(1, 3, 1, 1))

# 방법 2: 첫 층을 한 채널짜리로 바꾸고, 세 색의 가중치를 더해 넣는다
새_첫층 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
with torch.no_grad():
    새_첫층.weight.copy_(모델.conv1.weight.sum(dim=1, keepdim=True))   # (64, 3, 7, 7) → (64, 1, 7, 7)
모델.conv1 = 새_첫층
```

```
conv1 가중치: (64, 3, 7, 7)
conv1 가중치: (64, 1, 7, 7)
두 방법의 출력이 같은가: True
```

```
a × w1 + a × w2 + a × w3
= a × (w1 + w2 + w3)
```

대부분은 **방법 1**로 충분하다. 어느 쪽이든 정규화는 세 채널로 만든 뒤 ImageNet 값으로

# 10.8 저장하기: 전체냐 머리만이냐

```
# 10.4절처럼 특징 추출로 머리를 학습해 완성 모델을 만든다
모델.fc = nn.Identity()
학습_특징, _ = 특징_뽑기(모델, 로더_만들기(학습_사진, 학습_정답), 장치)
모델.fc = 머리_학습(학습_특징, 학습_정답)

torch.save(모델.state_dict(), "eurosat_resnet18.pt")      # 모델 전체
torch.save(모델.fc.state_dict(), "eurosat_head.pt")       # 머리만
```

**eurosat_resnet18.pt · 모델 전체**

**43,755.1KB**

**eurosat_head.pt · 머리만**

**21.9KB**

2,000배 차이. 특징 추출이면 몸통은 torchvision에서 언제든 다시 받으니 **머리만 저장해도 충분**하다

# 10.8 불러오기: 같은 설계도부터

```
# 불러오기 ①: 설계도를 그대로 만들지 않으면
새모델 = models.resnet18(weights=None)
새모델.load_state_dict(torch.load("eurosat_resnet18.pt", weights_only=True))
# 불러오기 ②: 머리를 바꾼 설계도에 전체를 채운다
새모델.fc = nn.Linear(512, 10)
새모델.load_state_dict(torch.load("eurosat_resnet18.pt", weights_only=True))
# 불러오기 ③: 몸통은 torchvision에서 받고 머리만 채운다
가벼운모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
가벼운모델.fc = nn.Linear(512, 10)
가벼운모델.fc.load_state_dict(torch.load("eurosat_head.pt", weights_only=True))
```

```
오류: size mismatch for fc.weight: copying a param with shape torch.Size([10, 512]) from checkpoint,
     the shape in current model is torch.Size([1000, 512]).
전체를 불러온 모델 87.0%
머리만 불러온 모델 87.0%
```

24차시 · 10.8 저장하고 불러오기 · 예제 10-transfer-learning/b20_save.py (핵심 부분, ①은 try 안에서 실행, 긴 오류 줄은 나눠 싣고 fc.bias 줄은 생략)

# 직접 실행 10분

1. b18_heads.py를 복사해 모델 목록을 ["resnet50", "densenet121", "convnext_tiny", "swin_t"]로 바꾼다. 머리는 어디에 있고, 특징은 몇 개가 들어오나?
2. b20_save.py 실행을 걸어 두고(26초), 폴더에 생긴 두 .pt 파일의 크기를 비교한다
3. b23_gray.py를 실행하고, 두 방법이 같은 이유를 옆 사람에게 식으로 설명한다

```
for 이름 in ["resnet50", "densenet121", "convnext_tiny", "swin_t"]:
```

24차시 · 직접 실행 · 복사본 3초(weights=None이라 내려받지 않음), b20 26초, b23 1초

# 미세조정한 모델을 나중에 다시 쓰려면 무엇을 저장해야 할까요?

Q

**가** 머리(fc)의 state_dict만

**나** 모델 전체의 state_dict

**다** 아무것도. 언제든 다시 받는다

**라** 학습률과 에폭 수만

# 24차시 정리

**머리 바꾸기**: named_modules()에서 마지막 nn.Linear를 찾아 rpartition · get_submodule · setattr

**모델 고르기**도 데이터로: ImageNet 순위가 우리 데이터에서 뒤집혔다. 흑백은 세 채널로 복사

**저장**: 머리를 바꾼 설계도를 똑같이 만든 뒤 load_state_dict. 특징 추출이면 머리만, 미세조정이면 전체

다음 차시: 데이터 양을 바꿔 세 방법을 한 표로 · 시험 데이터로 마지막 확인

25차시

25

# 실습: 데이터가 적을수록 전이 학습이 빛난다

책 10.9절 · 5일 과정 마무리

**학습 목표**

- 데이터 양을 바꿔 세 방법을 한 표로 비교한다
- 고치는 횟수를 맞추는 법과 itertools.cycle의 함정을 안다
- 시험 데이터를 딱 한 번 써서 최종 성적과 헷갈린 짝을 본다

# 10.9 데이터 양을 바꿔 가며 비교하기

### 처음부터

무작위 가중치 ResNet18
lr=0.001

배치 **240번** 고치기

### 특징 추출

특징을 뽑아 두고
머리만

**100에폭**

### 미세조정

몸통 lr=0.0001
머리 lr=0.001

배치 **64번** 고치기

클래스당 5장이면 한 에폭이 배치 1개, 100장이면 16개. 에폭이 아니라 **배치를 꺼내 고치는 횟수**를 맞춘다

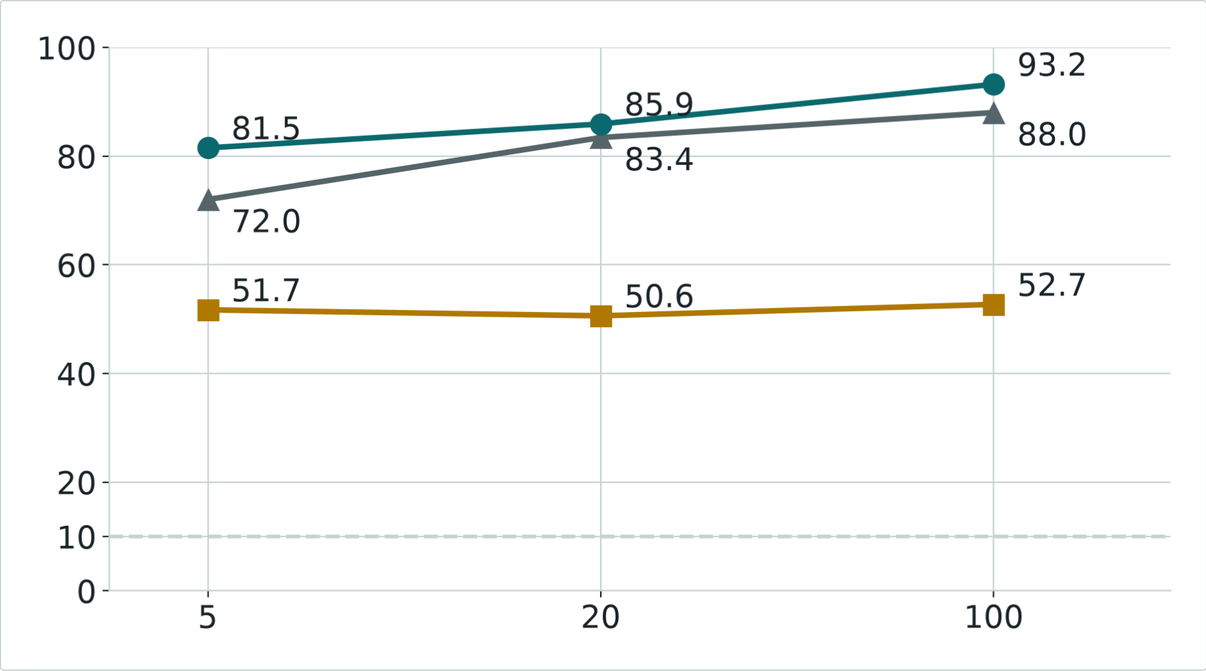
# 10.9 정해진 횟수만큼 고치기

```
def 끝없이(로더):
    """로더를 다 돌면 처음부터 다시 돈다. 돌 때마다 새로 섞고 새로 증강한다."""
    while True:
        yield from 로더
def 고치기(모델: nn.Module, 옵티마이저: torch.optim.Optimizer, 학습_로더, 횟수: int) -> None:
    """데이터 양과 상관없이 정해진 횟수만큼 배치를 꺼내 고친다."""
    모델.train()
    for x, y in itertools.islice(끝없이(학습_로더), 횟수):
        x, y = x.to(장치), y.to(장치)
        옵티마이저.zero_grad()
        nn.functional.cross_entropy(모델(x), y).backward()
        옵티마이저.step()
```

**itertools.cycle은 함정**: 첫 바퀴 배치를 저장해 **똑같이** 되풀이한다. 섞기도 증강도 첫 바퀴뿐

**while True: yield from**: 바퀴마다 로더를 새로 돌리니 매번 새로 섞고 **새로 증강**한다

# 10.9 데이터가 적을수록 차이가 크다




● 미세조정 · ▲ 특징 추출 · ■ 처음부터 · 가로 클래스당 장수 · 점선 10%

| 클래스당 | 처음부터 | 특징 추출 | 미세조정 |
|---|---|---|---|
| 5장 | 51.7% | 72.0% | 81.5% |
| 20장 | 50.6% | 83.4% | 85.9% |
| 100장 | 52.7% | 88.0% | 93.2% |

사진 **50장**으로 열에 여덟 넘게
미세조정 81.5%

5장에서도 미세조정이 9.5%p 앞선다
특징 추출은 **증강을 못 쓴다**

25차시 · 10.9 데이터 양을 바꿔 가며 비교하기 · 예제 10-transfer-learning/b21_sweep.py (출력, 그림은 같은 설정으로 다시 돌려 그린 것)

# 10.9 표를 읽고 나서

### 처음부터는 왜 50% 초반에?

데이터가 스무 배로 늘어도 고치는 횟수는 240번 그대로. 늘어난 데이터를 다 소화하지 못했다

무작위에서 시작한 모델은 끝나는 순간에 따라 크게 흔들린다 (23차시 그림: 20에폭 63.4%, 25에폭 69.9%, 30에폭 68.1%)

### 실무에서 무난한 순서

① 특징 추출로 빠르게 기준을 잰다

② 시간이 허락하면 미세조정으로 더 올린다

미세조정은 계산이 훨씬 많고, 학습률을 잘못 잡으면 물려받은 눈을 망가뜨린다

전이 학습은 **적은 데이터와 짧은 시간**, 두 가지를 한꺼번에 아껴 준다

# 10.9 시험 데이터로 마지막 확인

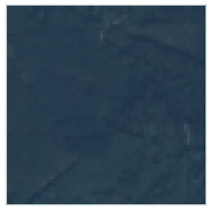
**시험 정확도 · 딱 한 번**

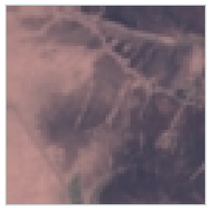
# 92.9%

검증 93.2%와 거의 같다

고른 방법: 클래스당 100장 + 미세조정
몸통 0.0001 · 머리 0.001 · 64번


경작지
**90%**


숲
**97%**


초지
**87%**


고속도로
**93%**

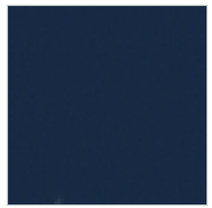

산업지역
**99%**


목초지
**92%**

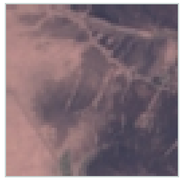

과수원
**89%**


주거지
**95%**

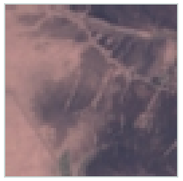

강
**93%**


바다·호수
**94%**

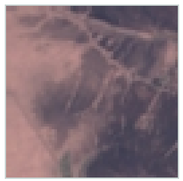
# 10.9 가장 많이 헷갈린 짝


→


정답 초지 → 예측 숲
**6번**


→


정답 과수원 → 예측 초지
**6번**


→


정답 초지 → 예측 과수원
**5번**

모두 **식물로 덮인 땅**끼리
64 × 64 한 장에서는 사람이 봐도 애매하다

```
# (정답, 예측) 칸마다 개수
혼동 = torch.bincount(
    시험_정답 * 10 + 예측,
    minlength=100).reshape(10, 10)
```

25차시 · 10.9 시험 데이터로 마지막 확인 · 예제 10-transfer-learning/b22_final.py (출력과 핵심 부분, 사진은 클래스 예시, 코드는 한 줄을 나눠 실음)

# 직접 실행 10분

1. b22_final.py를 실행해 시험 정확도 92.9%와 헷갈린 짝을 확인한다
2. 22차시 특징 추출에서는 입력을 160으로 키우면 87.0 → 89.1%였다. 미세조정에서도 그럴까? b22를 복사해 시험 대신 **검증** 데이터로, 크기 112와 160을 비교한다
3. 결과를 보고 "손잡이마다 효과는 검증으로 확인한다"를 자기 말로 적는다

```
로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True, 크기=160)
```

# 클래스당 5장뿐일 때, 미세조정이 특징 추출보다 9.5%p 앞선 까닭은?

Q

**가** 미세조정이 에폭을 더 많이 돌아서

**나** 특징 추출은 증강을 쓸 수 없어서

**다** 미세조정만 ImageNet 가중치를 써서

**라** 특징 추출은 검증 데이터를 못 봐서

# 25차시 정리

데이터가 적을수록 차이가 크다: 클래스당 5장에서 처음부터 51.7% · 특징 추출 72.0% · **미세조정 81.5%**

고정 횟수로 반복할 때는 while True: yield from. **itertools.cycle**은 증강을 막는다

검증으로 고르고 **시험은 한 번**: 92.9%. 헷갈린 짝은 식물로 덮인 땅끼리

이어서: 5일 과정을 한 장에 모아 본다

# 5일 동안 쌓아 올린 것

### 1일차

1~5차시 · 4~5장

텐서의 모양과 장치
기울기와 경사하강법
autograd

### 2일차

6~10차시 · 6~7장

nn.Module
표준 학습 루프
Dataset · DataLoader
transforms

### 3일차

11~15차시 · 7~8장

분할과 샘플러
과적합 진단과 대책
배치 정규화
스케줄러

### 4일차

16~20차시 · 9장

합성곱 · 채널 · 풀링
첫 CNN
작은 ResNet

### 5일차

21~25차시 · 10장

사전학습 모델
특징 추출 · 미세조정
저장과 불러오기

모델이 MLP에서 CNN, 남의 ResNet18로 바뀌어도 **학습하는 모양은 같았다**: 기울기 지우기, 순전파, 손실 계산, 역전파, 파라미터 갱신

# 5일 과정을 마치며

1. 모델은 **텐서와 기울기**로 움직인다
2. 어떤 모델이든 **같은 학습 루프**로 학습한다
3. 학습 점수가 아니라 **검증 점수**를 본다
4. 사진은 **합성곱**으로 무늬를 찾는다
5. 데이터가 적으면 **남이 학습한 눈**을 빌린다

오류 없이 **조용히 틀리는** 곳(전처리, 학습률, train/eval)일수록 숫자로 확인한다

## 책에서 이어지는 길

- 11장 RNN과 LSTM
- 12장 어텐션과 트랜스포머
- 13~14장 한국어 NLP, LLM과 LoRA
- 18장 실험 관리와 재현성
- 19~20장 경량화와 모델 서빙